

PROJECT EXPLAINER

High-Throughput Go and Python Microservices on Oracle Kubernetes Engine

Explained in simple terms: every component, every workflow, and the architecture end to end

Item	Detail
Subject of this document	Deployment of high-throughput Go and Python microservices on Oracle Kubernetes Engine (OKE), running on Oracle Cloud Infrastructure (OCI).
OCI services involved	OKE, Compute, IAM, Object Storage, Virtual Cloud Networks (VCN), Load Balancers, Container Registry (OCIR), Monitoring and Logging.
Kubernetes features involved	Node pools, horizontal pod autoscaling, cluster autoscaling, rolling deployments, liveness / readiness / startup probes, workload isolation, resource requests and limits.
Security model	Instance-principal authentication with least-privilege IAM policies. No passwords, keys or tokens stored in application code.
Who this document is for	Anyone who wants to understand the project without prior Kubernetes or cloud experience: reviewers, interviewers, new joiners, and non-technical stakeholders.
How to read it	Start with Section 1 for the plain-English summary, then Section 2 which decodes the original one-line project description word by word. Sections 5 to 7 explain each component. Sections 8 to 10 walk through the workflows.

Note: configuration values shown in this document (node shapes, replica counts, CIDR ranges, thresholds) are representative examples used to make the explanation concrete.

Contents

The document runs from the plainest possible summary to the fullest technical detail. Read straight through, or jump to the section you need.

Section	What is inside it
1. The Project in Plain English	The one-paragraph version, an everyday analogy, the problem being solved, and what "high-throughput" actually means here.
2. The One-Line Description, Decoded	Every phrase of the original project summary translated term by term.
3. Glossary	Every term used anywhere in this document, in one line each: container and Kubernetes terms, OCI terms, scaling and release terms.
4. The Big Picture	Diagram 1 — the whole system in a single view, with a five-point reading guide.
5. Every Component, Explained End to End	Fifteen components, each on the same five questions: the Go services, the Python services, containers and OCIR, OKE, Compute, the VCN, the Load Balancer, node pools, autoscaling, rolling deployments, health probes, workload isolation, IAM and instance principals, Object Storage, and observability.
6. The Architecture Diagram	Diagram 2 — the full architecture, then every element of it explained from the outside inwards.
7. Zooming In on One Machine	Diagram 3 — one worker node, one pod, one container: requests and limits, and the three probes on a real timeline.
8. Workflows, End to End	Seven workflows step by step: serving a request, releasing a version, a traffic spike, a machine dying, obtaining permission to touch data, upgrading the cluster, and changing a setting.
9. Design Decisions and Trade-Offs	Why each choice was made, what the alternative would have cost, and the honest limitations of this design.
10. Quick Answers and a One-Page Summary	Likely questions with short answers, and the whole project condensed onto one page.
Appendix A. Large-Format Diagrams	Four full-width landscape diagrams for printing or presenting: the complete architecture, the request path, the release pipeline with the rolling deployment, and autoscaling with self-healing.

1. The Project in Plain English

1.1 The one-paragraph version

This project took a set of small, independent programs — some written in Go, some in Python — and put them to work on Oracle's cloud in a way that can handle a very large number of requests without falling over. Instead of installing the programs directly onto servers, each one was packaged into a container: a sealed box that holds the program and everything it needs to run. Those containers are then handed to Kubernetes, a system whose entire job is to keep the right number of copies running, restart anything that dies, and move work around when a machine fails. Oracle runs the Kubernetes brain for you — that managed offering is called Oracle Kubernetes Engine, or OKE. Around that core, the project set up the supporting pieces: a private network with carefully controlled doors, a load balancer that acts as the single public front entrance, fleets of virtual machines that grow and shrink with traffic, a storage bucket for files, and an identity system that lets the machines prove who they are without anyone ever writing a password into the code.

1.2 The everyday analogy

Picture a busy restaurant chain.

- The containers are identical meal kits. Whatever kitchen you open one in, it behaves exactly the same. That is what makes copies interchangeable.
- Kubernetes (OKE) is the operations manager. It counts how many kitchens are open, notices when one catches fire, opens a replacement, and never lets the chain run below the agreed minimum.
- The node pools are two separate kitchen buildings. One is fitted out for fast short-order cooking (the Go services), the other for slow heavy prep work (the Python services). Keeping them in separate buildings means a rush in one cannot take the ovens away from the other.
- The load balancer is the head waiter at the door. Customers only ever speak to the head waiter, who knows which kitchens are healthy today and sends each order to one of them in turn.
- Autoscaling is calling in extra staff when a coach party arrives and sending them home when it is quiet, so you are not paying for idle hands.
- Health probes are the manager walking past each kitchen. "Are you awake?" "Are you ready for orders?" A kitchen that stops answering is quietly taken off the order list and restarted.
- Rolling deployments are changing the menu one kitchen at a time, so the restaurant never closes.
- Instance principals are staff badges that the building issues automatically and that expire the same day, instead of a master key that everyone copies and nobody ever changes.

1.3 What problem was actually being solved

Three problems, really, and every design choice in the project traces back to one of them.

Problem	What it looks like when unsolved	How this project answers it
Scale	Traffic spikes and the service slows to a crawl or falls over, because there is a fixed number of servers.	Two layers of automatic scaling: more copies of the program first, then more machines to put them on.
Reliability	One crashed process, one bad release, or one dead machine takes the whole service down.	Many interchangeable copies, automatic health checking and restarting, and releases that

Problem	What it looks like when unsolved	How this project answers it
		replace copies one at a time.
Security and cost control	Cloud credentials sit in config files and leak; every service can reach everything; unused machines run all month.	Machines authenticate as themselves with short-lived certificates, policies grant only the exact permissions needed, and idle machines are removed automatically.

1.4 What "high-throughput" means here

High-throughput means the system is built to process a large number of requests per second, continuously, rather than to handle one big job. That shapes the design in specific ways:

- Go is used on the hottest paths because it is compiled, starts almost instantly, and handles thousands of concurrent connections in a small amount of memory.
- Python is used where developer speed and its data and integration libraries matter more than raw request rate; because a Python process is heavier, it is given its own machines so it cannot crowd out the Go services.
- Nothing important is stored on any individual machine. Any copy can serve any request, which is what makes it safe to add and remove copies freely.
- Every copy declares how much CPU and memory it needs up front, so the system can pack machines efficiently and know exactly when it has run out of room.

2. The Original One-Line Description, Decoded

The project was originally summarised in a single dense sentence. This section takes that sentence apart phrase by phrase. Each phrase is explained again in far more depth later in the document — this table is the map.

Phrase in the original line	What it literally means	Why it was worth mentioning
Deployed high-throughput ... microservices	Put many small independent programs into live service, built to handle a heavy and continuous flow of requests.	It signals scale. A design for ten requests a minute looks nothing like a design for thousands a second.
GoLang and Python	Two programming languages were used side by side, each chosen for what it is good at.	It shows the platform is language-agnostic and that the two very different resource profiles were handled deliberately.
Oracle Kubernetes Engine (OKE)	Oracle's managed Kubernetes service. Oracle runs and patches the control plane; you supply and pay for the worker machines.	Managed control plane means no time spent maintaining the cluster brain, and no single point of failure you have to build yourself.
OCI Compute	The virtual machines that the containers actually run on. These become the Kubernetes worker nodes.	This is where the CPU and memory — and most of the bill — actually live.
IAM	Identity and Access Management: who is allowed to do what to which resource.	It is the difference between a leaked file and a locked one, and it is the part most often done badly.
Object Storage	A bucket for files, addressed by name over the network, with effectively unlimited capacity.	Containers are disposable, so anything that must survive has to live outside them.
VCNs	Virtual Cloud Networks: your own private, isolated network inside the cloud, cut into subnets with controlled doors.	It is the boundary that keeps the worker machines unreachable from the internet.
Load Balancers	The single public entry point that spreads incoming requests across all the healthy copies.	It is what makes many copies look like one service, and what makes zero-downtime releases possible.
node pools	Named groups of identical worker machines, each group with its own size, shape and scaling rules.	This is the mechanism behind workload isolation and behind scaling the two languages independently.
autoscaling	Automatically changing the number of running copies, and the number of machines, based on live demand.	It handles traffic spikes without a human, and it stops you paying for capacity you are not using.
rolling deployments	Replacing the old version with the new one a few copies at a time instead of all at once.	Users never see downtime, and a bad release stops itself part-way through instead of taking everything down.

Phrase in the original line	What it literally means	Why it was worth mentioning
health probes	Small automated checks the platform runs against every copy to ask whether it is alive and whether it is ready for traffic.	Without them, the platform cannot tell a hung process from a healthy one, and every other safety feature stops working.
workload isolation	Rules that guarantee particular workloads land on particular machines and cannot interfere with each other.	It stops a heavy Python job from starving the latency-sensitive Go services of CPU.
instance-principal least-privilege access	Each machine proves its own identity to the cloud using a short-lived certificate, and is granted only the minimum permissions it needs.	No long-lived secrets in code or config, and a compromise of one machine gives an attacker almost nothing.

3. Glossary — Every Term Used in This Document

Read this once and the rest of the document becomes straightforward. Terms are grouped by theme rather than alphabetically, because they make more sense together.

3.1 Containers and Kubernetes

Term	Simple explanation
Container	A sealed box holding a program plus every library and file it needs. It runs identically on a laptop, in a test environment and in production, which removes the whole class of "it worked on my machine" problems.
Image	The saved, read-only blueprint of a container. You build an image once, give it a version tag, and then start as many identical containers from it as you like.
Registry (OCIR)	The warehouse that stores images. Machines pull images from it when they need to start a container.
Pod	The smallest thing Kubernetes runs. In practice it is a thin wrapper around one container. Kubernetes never manages containers directly — it manages pods.
Node (worker node)	One virtual machine that runs pods. It contributes CPU and memory to the cluster.
Cluster	The control plane plus all the worker nodes, treated as a single pool of computing capacity.
Control plane	The brain of the cluster. It stores the desired state, decides which pod goes on which node, and continuously corrects reality until it matches. In OKE, Oracle runs this for you.
kubelet	The agent on each node. It starts and stops containers, runs the health probes, and reports node and pod status back to the control plane.
kube-proxy	The networking agent on each node. It makes sure traffic addressed to a Service actually reaches a live pod.
Deployment	The object you actually write and edit. It says: run this image, this many times, with these settings. Everything else follows from it.
ReplicaSet	Created automatically by a Deployment. Its single job is to keep exactly N pods of one specific version alive. Each new version gets a new ReplicaSet, which is what makes rollback instant.
Service	A stable internal name and IP address for a group of pods. Pods come and go constantly; the Service address never changes.
Namespace	A folder inside the cluster used to group related objects and apply quotas and access rules to them.
ConfigMap	A place to keep non-secret settings — log level, timeouts, feature flags — outside the image, so the same image can be used in every environment.
Secret	The same idea for sensitive values such as database passwords and API tokens, stored and mounted separately from ordinary configuration.

3.2 Oracle Cloud Infrastructure

Term	Simple explanation
Region	A geographic location containing Oracle data centres, for example us-ashburn-1. All the resources in this project live in one region.
Availability domain	An independent data centre inside a region. Spreading copies across availability domains means one data centre failing does not take out the service.
Fault domain	An independent group of hardware inside one availability domain. A second, finer level of spreading.
Tenancy	Your whole account in Oracle Cloud.
Compartment	A labelled box inside the tenancy used to group resources for access control, budgeting and tidiness. This project uses a dedicated production compartment.
Compute instance	A virtual machine. In this project, every worker node is a compute instance.
Shape	The size of a virtual machine — how many cores and how much memory. A flexible shape lets you choose the exact numbers rather than picking from fixed sizes.
VCN	Virtual Cloud Network: your own private network in the cloud, with an address range you choose.
Subnet	A slice of the VCN address range. Public subnets can hold resources reachable from the internet; private subnets cannot.
Gateway	A controlled door out of the VCN. Different gateways lead to different places, which is the whole point.
Security list / NSG	Firewall rules. A security list applies to a whole subnet; a network security group applies to a chosen set of resources, which is more precise.
Load balancer	A managed service that owns a public address, checks which backends are healthy, and distributes incoming requests across them.
Object Storage	File storage addressed over the network. Files are called objects and live in a bucket.
IAM policy	A written rule granting a specific group permission to perform specific actions on specific resources.
Dynamic group	A group whose members are cloud resources, not people. Membership is defined by a matching rule, so a machine created five minutes ago is already a member.
Instance principal	The mechanism that lets a compute instance authenticate to OCI as itself, using a short-lived certificate it fetches locally. This is what removes stored credentials.

3.3 Scaling, health and release terms

Term	Simple explanation
Node pool	A named group of identical worker machines with its own shape, size limits and scaling rules. A cluster can have several.
Replica	One running copy of a service. Ten replicas means ten pods doing the same job behind one address.
Horizontal Pod Autoscaler (HPA)	Watches a metric such as average CPU and changes the number of replicas to keep that metric near a target.
Cluster Autoscaler	Watches for pods that cannot be placed because no machine has room, and adds machines. It also removes machines that are no longer needed.
Requests and limits	Requests are what a container reserves and what the scheduler uses when deciding where it fits. Limits are the hard ceiling it may not exceed.
Liveness probe	Asks "are you still alive?" Repeated failure means the container is restarted.
Readiness probe	Asks "can you take traffic right now?" Failure means the pod is removed from the load balancer but left running.
Startup probe	Asks "have you finished booting?" It suspends the other two probes until the app is up, so a slow start is not mistaken for a crash.
Rolling deployment	Replacing old pods with new ones gradually so the service stays available throughout.
maxSurge / maxUnavailable	The two dials that control a rolling update: how many extra pods may exist temporarily, and how many may be missing at once.
Taint and toleration	A taint is a "keep off" sign on a node. A toleration is the permit that lets a specific pod ignore that sign. Together they reserve machines for particular workloads.
Node label / node selector	A label is a tag on a node; a selector is a pod saying which tags it needs. This is how a pod is steered to the right pool.
Affinity / anti-affinity	Rules asking for pods to be placed near each other, or deliberately apart. Anti-affinity is used here to spread replicas across machines.
PodDisruptionBudget	A promise that a minimum number of pods will stay running during any voluntary disruption, such as a node drain or an upgrade.
Drain / cordon	Cordon marks a node as closed to new pods. Drain politely evicts the pods already on it. Together they let a machine be removed without dropping requests.

4. The Big Picture

Before going into detail, here is the whole system in one view. Everything that follows is an expansion of one of these boxes.

Diagram 1 - The Big Picture in One Look

Read left to right: a request arrives, it is spread out, it runs inside a container, and files land in storage.

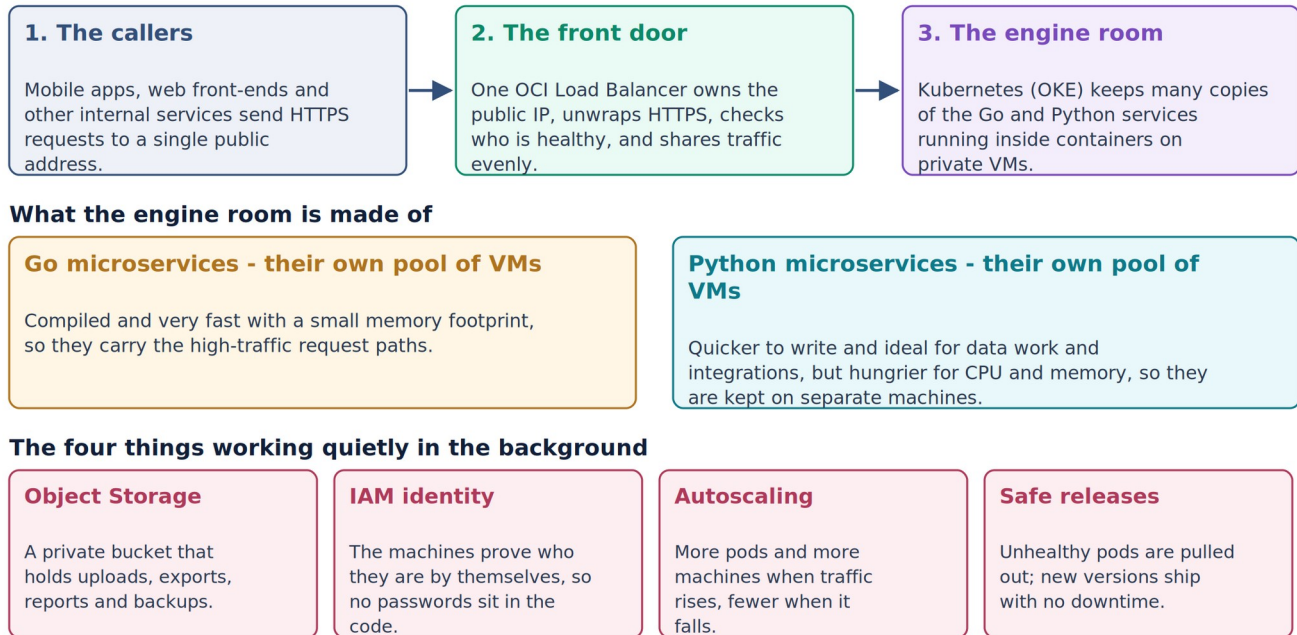


Diagram 1 — the shape of the whole system in a single view.

4.1 Reading Diagram 1

1. The callers. Mobile apps, web front-ends and other internal systems all send ordinary HTTPS requests to one public address. They know nothing about Kubernetes, containers or how many machines exist.
2. The front door. A single OCI Load Balancer holds the public IP address for that name. It unwraps the encryption, checks which copies of the service are healthy right now, and passes each request to one of them.
3. The engine room. Behind that door sits the OKE cluster: many identical copies of the Go and Python services, running as containers on a fleet of private virtual machines.
4. The two workload families. The Go services and the Python services live on separate groups of machines. They scale separately and cannot steal resources from each other.
5. The four background systems. Object Storage keeps the files. IAM handles identity. Autoscaling adjusts capacity. Health probes and rolling updates keep the service correct while it changes.

The single most important idea in this whole document

Nothing in the running system is precious. Every pod and every machine is disposable and replaceable.

Because any copy is as good as any other, the platform is free to kill a sick one, add ten more during a rush, or replace them all with a new version — without asking anyone and without interrupting the service. Every feature described in the rest of this document exists to make that freedom safe.

5. Every Component, Explained End to End

Each component below follows the same five questions: what it is, what it does in this project, which settings were chosen, why it matters, and what would break without it.

5.1 The Go microservices

Go is a compiled language. The build step turns the source code into a single self-contained executable file, which means the container image can be tiny and the program can start in a fraction of a second.

Aspect	Explanation in simple terms
What it is	Small independent programs written in Go, each responsible for one business capability — for example placing an order, or looking up a price. Each is built into its own container image.
What it does in this project	They carry the request paths that see the most traffic and the tightest latency requirements. Go handles thousands of simultaneous connections cheaply because its concurrency units (goroutines) are far lighter than operating-system threads.
Settings that were chosen	Multi-stage container build producing a minimal image of a few tens of megabytes; typically 500 millicores of CPU and 512 MiB of memory requested per pod, with a ceiling of 1500 millicores and 1 GiB; graceful shutdown handling so in-flight requests finish when the pod is asked to stop.
Why it matters	A small image pulls and starts quickly, which is exactly what you need when the autoscaler is adding pods during a traffic spike. Low memory per request means more requests fit on the same machine, which directly lowers cost.
What would break without it	The same traffic would need noticeably more machines, and scaling up would be slower because every new pod would take longer to become useful.

5.2 The Python microservices

Python is interpreted rather than compiled. It is slower per request and heavier in memory, but it is much faster to write and it has the strongest ecosystem for data processing, reporting and third-party integrations.

Aspect	Explanation in simple terms
What it is	Independent Python services — typically Flask or FastAPI applications served by Gunicorn with Uvicorn workers — packaged the same way as the Go services.
What it does in this project	They handle work where flexibility and library support matter more than raw request rate: report generation, data transformation, integration with external systems, scheduled batch work.
Settings that were chosen	Around 1000 millicores of CPU and 2 GiB of memory requested per pod with a ceiling above that; a fixed worker count per pod tuned to the CPU limit; a startup probe with a generous allowance because a Python application typically takes several seconds to import its dependencies and become ready.
Why it matters	Giving Python honest, larger resource requests and its own machines means its heavier appetite is planned for rather than discovered during an incident.

Aspect	Explanation in simple terms
What would break without it	Python pods would land on the same machines as the Go pods, and a burst of report generation would consume the CPU that the latency-sensitive Go services depend on.
Why two languages at all?	Because the cost of running both is almost zero once everything is containerised. The platform does not know or care what is inside a container — it only sees CPU, memory and health. That is precisely what makes it reasonable to pick the best language for each job instead of forcing one language everywhere.

5.3 Containers and the image registry (OCIR)

Aspect	Explanation in simple terms
What it is	A container image is the read-only blueprint of a running container. OCIR — the OCI Container Registry — is the private warehouse where those images are stored and versioned.
What it does in this project	Every build of every service produces an image, tagged with the exact version or commit, and pushed to OCIR. When a pod starts, the node pulls that exact image and runs it.
Settings that were chosen	Multi-stage builds so that compilers and build tools never reach the final image; images run as a non-root user with a read-only root filesystem; immutable version tags rather than a moving "latest" tag; nodes pull from OCIR through the Service Gateway, so image pulls never traverse the internet.
Why it matters	An immutable, versioned image is what makes a deployment reproducible and a rollback trustworthy. If version 2.4.1 is broken, going back to 2.4.0 gives you exactly what was running before, byte for byte.
What would break without it	"Latest" tags mean two pods claiming to run the same version can be running different code, which makes incidents very hard to reason about.

5.4 Oracle Kubernetes Engine (OKE)

Kubernetes is a control loop. You describe the state you want; it continuously compares that to reality and takes whatever action closes the gap. OKE is Oracle's managed version of it.

Aspect	Explanation in simple terms
What it is	A managed Kubernetes cluster. Oracle operates the control plane — the API server, the scheduler, the controllers and the etcd datastore — across multiple availability domains. You provide the worker nodes.
What it does in this project	It is the component that decides which pod runs on which machine, notices when something has died, restarts it, and orchestrates version changes. It is the manager in the restaurant analogy.
Settings that were chosen	A private Kubernetes API endpoint so the cluster cannot be administered from the open internet; worker nodes in private subnets; multiple node pools; the cluster autoscaler and metrics server installed as cluster add-ons.
Why it matters	Running your own Kubernetes control plane is a genuinely difficult, permanent job — highly available etcd, certificate rotation, version upgrades. Using OKE removes that work entirely, and

Aspect	Explanation in simple terms
	Oracle does not charge for the control plane itself.
What would break without it	Either a large amount of engineering time goes into cluster maintenance, or a home-built system ends up reimplementing scheduling, health checking and rolling updates badly.

The three things Kubernetes does that are hard to appreciate until they are gone

- Desired state. You never say "start a pod". You say "there should be six", and the system takes responsibility for making that true, permanently, without you watching.
- Bin packing. The scheduler solves the puzzle of which pod fits on which machine, using each pod's declared CPU and memory requests, every time anything changes.
- Reconciliation. Every failure — a crashed process, a dead machine, an evicted pod — is handled by the same loop rather than by a special case. That is why the system behaves predictably under conditions nobody anticipated.

5.5 OCI Compute — the worker nodes

Aspect	Explanation in simple terms
What it is	Ordinary virtual machines. Each one joins the cluster as a worker node and contributes its CPU and memory to the shared pool.
What it does in this project	They are where the containers actually execute. Each node runs the kubelet (the local agent), kube-proxy (local networking) and containerd (the container runtime), plus every pod scheduled onto it.
Settings that were chosen	Flexible shapes such as VM.Standard.E4.Flex, letting cores and memory be chosen to match the workload — smaller nodes for the Go pool, larger for the Python pool; nodes booted from the Oracle-managed OKE worker image so the kubelet version always matches the cluster; nodes placed in private subnets with no public IP address; nodes spread across availability domains.
Why it matters	This is where nearly all the cost sits, and where the CPU and memory limits are ultimately felt. Choosing the shape well is the difference between a well-packed cluster and one paying for air.
What would break without it	There would be nothing to run the containers on. Choosing shapes badly leads to stranded capacity — for example nodes with plenty of unused memory but no free CPU.
Why the worker nodes have no public IP address It removes an entire category of attack. A machine with no route in from the internet cannot be scanned, brute-forced or reached directly at all. It can still reach out — through the NAT Gateway for internet destinations and the Service Gateway for Oracle services — and it can still receive traffic, but only from the load balancer, and only on the specific ports the security rules allow.	

5.6 The Virtual Cloud Network (VCN)

The VCN is the private network that everything else lives inside. It is worth spending time on, because it is what turns a pile of cloud resources into a defensible system.

Aspect	Explanation in simple terms
What it is	A private, isolated network inside the OCI region with an address range you choose — for example 10.0.0.0/16, which provides roughly 65,000 private addresses.
What it does in this project	It divides the system into zones with different exposure and controls every route between them. Public-facing things go in one place, private things in another, and traffic between them passes only through rules you wrote.
Settings that were chosen	A public subnet (10.0.1.0/24) holding only the load balancer; a private subnet (10.0.2.0/24) holding all worker nodes; a private subnet (10.0.3.0/24) for the Kubernetes API endpoint; regional subnets so resources can sit in any availability domain; route tables and security rules per subnet.
Why it matters	Layout is security. Splitting into subnets means the blast radius of any mistake is limited to one zone, and it makes the firewall rules simple enough to actually be correct.
What would break without it	Everything sits in one flat network. One exposed component means everything is reachable, and firewall rules become an unreviewable list of exceptions.

5.6.1 The three gateways, and why there are three

Gateway	Direction of travel	What it is for and why it exists separately
Internet Gateway	In and out	The two-way door. It is what makes the load balancer reachable from the public internet. Only the public subnet routes through it — the worker nodes never do.
NAT Gateway	Outbound only	The one-way door. Private nodes use it to fetch operating-system patches, language packages and third-party APIs. Return traffic for those conversations is allowed, but nobody on the internet can start a conversation inwards.
Service Gateway	To OCI services only	The private lane. Calls to Object Storage and OCIR travel across Oracle's own backbone and never touch the internet at all. This is faster, cheaper (no NAT data charges) and materially more secure, because that traffic is never exposed.

The distinction matters. A common mistake is to route everything through a NAT Gateway because it works. It does work — but every byte written to Object Storage then travels out to the internet and back, which is slower, costs more, and needlessly widens the exposure of your data path.

5.6.2 Security lists and network security groups

These are the firewall rules. A security list applies to every resource in a subnet; a network security group applies to a specific set of resources, which allows tighter rules. The project used both, following a deny-by-default posture:

- Inbound port 443 from anywhere is allowed — but only to the load balancer, and nothing else in the public subnet accepts traffic.
- The load balancer may reach worker nodes on the node-port range (30000–32767) and nowhere else.
- Worker nodes may talk to each other freely, because pod-to-pod traffic and Kubernetes internals depend on it.
- Outbound traffic is permitted to the NAT and Service Gateways.

- Anything not explicitly listed is denied. Rules are written by port and by source, never as blanket allowances.

5.7 The Load Balancer

Aspect	Explanation in simple terms
What it is	A managed OCI Flexible Load Balancer sitting in the public subnet. It owns the public IP address that the DNS name points to.
What it does in this project	It terminates HTTPS, decides which backend is healthy, and distributes each incoming request across the worker nodes. It is the only component that the outside world ever speaks to directly.
Settings that were chosen	A listener on port 443 with the TLS certificate managed in OCI (Vault / Certificates) rather than inside the application; a backend set containing the worker nodes on the service node port; an HTTP health check performing GET /healthz every 10 seconds with a small failure threshold; round-robin distribution; flexible bandwidth with a minimum and maximum so it can absorb bursts without being paid for constantly.
Why it matters	It is what makes many replicas look like a single service. It is also what makes zero-downtime releases possible, because it stops sending traffic to a backend the moment that backend stops passing its health check.
What would break without it	Clients would have to know the address of individual machines. Every scaling event and every release would be visible to users as errors.
Why terminate TLS at the load balancer Certificates are managed in one place instead of inside every container image, renewals do not require a redeployment, and the encryption and decryption work is done by the managed service rather than by your application's CPU. Traffic between the load balancer and the nodes stays inside the private network.	

5.8 Node pools

Aspect	Explanation in simple terms
What it is	A node pool is a named group of identical worker machines with its own shape, its own size limits, its own labels and taints, and its own scaling rules. One cluster can contain several.
What it does in this project	This project used at least two: one for the Go services and one for the Python services. All the machines in a pool are created from the same template, so any one of them is interchangeable with any other.
Settings that were chosen	Pool A — Go: medium shape (for example 4 cores, 32 GB), label workload=go-api, taint dedicated=go:NoSchedule, autoscaling between 2 and 6 nodes. Pool B — Python: larger shape (for example 8 cores, 64 GB), label workload=py-worker, taint dedicated=py:NoSchedule, autoscaling between 2 and 8 nodes. Both pools spread across availability domains.
Why it matters	Pools are the unit of isolation, the unit of scaling and the unit of upgrade. Because each pool has its own limits, one workload's growth can never consume the capacity another depends on.
What would break without it	One pool of general-purpose machines forces a compromise shape that suits neither workload, removes the ability to scale them independently, and makes an upgrade an all-or-nothing event.

5.9 Autoscaling

There are two independent scalers, working at two different levels. Understanding that they are separate — and that they must be in that order — is the key to this section.

5.9.1 Level one: the Horizontal Pod Autoscaler (more copies)

Aspect	Explanation in simple terms
What it is	A controller that watches a metric — typically average CPU across the pods of one Deployment — and changes the number of replicas to keep that metric near a target.
What it does in this project	When average CPU across the order-api pods climbs above the target, it increases the replica count. When it falls, it reduces it again, subject to a floor and a ceiling.
Settings that were chosen	Target of roughly 70% average CPU utilisation; a minimum of 3 replicas so a single failure never leaves the service thin; a maximum in the tens to cap runaway scaling and cost; a stabilisation window on scale-down so brief dips do not cause thrashing.
Why it matters	Pods start in seconds, so adding copies is by far the fastest way to absorb a surge. The floor of 3 also guarantees redundancy across machines at all times.
What would break without it	Traffic spikes would queue behind a fixed number of copies, latency would climb, and a human would have to notice and intervene.

5.9.2 Level two: the Cluster Autoscaler (more machines)

Aspect	Explanation in simple terms
What it is	A controller that watches for pods that cannot be scheduled because no node has enough free resources, and asks the node pool for additional machines. It also removes machines that are no longer needed.
What it does in this project	When the pod autoscaler asks for more replicas than the current machines can hold, the extra pods sit in Pending. The cluster autoscaler sees them and grows the node pool. Later, when nodes sit underused for a sustained period, it drains and deletes them.
Settings that were chosen	Configured per node pool with the minimum and maximum node counts described above; a scale-down delay so nodes are not removed the moment they look idle; respect for PodDisruptionBudgets so draining a node can never take the service below its minimum.
Why it matters	It is what makes the cluster genuinely elastic instead of merely flexible within a fixed fleet — and, just as importantly, it is what stops you paying for machines nobody is using.
What would break without it	Scaling would stop dead at the capacity of the machines you happened to have. Pods would sit Pending indefinitely, or you would over-provision permanently and pay for peak capacity around the clock.
Why the order matters	Pods first, machines second. Adding a pod takes seconds; booting a machine and having it join the cluster takes minutes. Scaling pods first absorbs the spike immediately and only escalates to new machines when the existing ones are genuinely full.

5.10 Rolling deployments

Aspect	Explanation in simple terms
What it is	A release strategy that replaces old pods with new ones gradually, a few at a time, rather than stopping everything and starting the new version.
What it does in this project	When a new image tag is applied, the Deployment creates a new ReplicaSet and shifts pods across in small steps. At every step the Service contains a mixture of old and new pods, all of them passing their readiness checks.
Settings that were chosen	maxSurge 1 (one extra pod may exist temporarily) and maxUnavailable 0 (capacity may never drop below the target); a minimum ready time so a pod must stay healthy for a period before it counts as successful; a progress deadline after which a stuck rollout is marked failed; revision history retained so rollback is a single command.
Why it matters	It removes the deployment window entirely. It also acts as an automatic circuit breaker: if the first new pod never passes its readiness probe, the rollout halts with the old version still serving every request.
What would break without it	Every release means downtime, and a bad release takes the entire service down at once instead of being caught after one pod.

This is worth stating plainly, because it is the point people most often miss: a rolling deployment is not just a convenience. It is the mechanism that converts a bad release from an outage into a non-event.

5.11 Health probes

Probes are how the platform forms an opinion about whether a pod is working. Everything else — load balancing, restarting, rolling updates, draining — depends on that opinion being accurate.

Probe	The question it asks	What happens, and what it must be careful about
Startup	Have you finished starting up?	It runs first and suspends the other two probes until it succeeds. Configured with a generous allowance, for example up to 30 attempts two seconds apart. This is what lets a slow-starting Python application boot without being mistaken for a crash and restarted forever.
Readiness	Can you take traffic right now?	Runs every few seconds. Failing removes the pod from the Service and from the load balancer's backend list, but does not restart it. This is the right response to a temporary condition such as a dependency being briefly unavailable, or a pod still warming its caches.
Liveness	Are you still alive at all?	Runs every ten seconds or so. After a small number of consecutive failures the container is restarted. It must test something genuinely local: if a liveness check calls a shared database, one slow database can trigger every pod in the fleet to restart simultaneously, turning a small problem into an outage.

The distinction that matters most

Readiness and liveness sound similar and are frequently confused, but they produce opposite behaviour. Readiness failure means "leave me running, just stop sending me work". Liveness failure means "I am beyond saving, restart me".

Getting them the wrong way round produces one of two classic failures: a pod that keeps receiving traffic it cannot serve, or a pod that is restarted every time a downstream dependency has a bad minute.

5.12 Workload isolation

Aspect	Explanation in simple terms
What it is	A set of scheduling rules that guarantee particular workloads land on particular machines, and that no workload can consume more than its share.
What it does in this project	It keeps the Go services and the Python services on separate node pools, and spreads the replicas of any one service across different machines.
Settings that were chosen	Node labels (workload=go-api, workload=py-worker) with matching node selectors on the pods; taints on each pool with matching tolerations, so a pod without the correct toleration cannot land there even by accident; pod anti-affinity so two replicas of the same service prefer different nodes; resource requests and limits on every container; separate namespaces with resource quotas.
Why it matters	It converts "they should not interfere with each other" from a hope into a guarantee enforced by the scheduler.
What would break without it	A heavy Python batch job would land beside the latency-critical Go services and consume the CPU they need. All replicas of a service could end up on one machine, so losing that machine would take the whole service down.

Labels versus taints — why both are used

A node selector is the pod choosing where to go. A taint is the node refusing anyone who has not been given explicit permission. Using only labels means a pod that forgets its selector lands anywhere; using only taints means a pod with a broad toleration could still drift onto the wrong pool. Together they make the placement mutual and therefore reliable.

5.13 IAM, instance principals and least privilege

This is the part of the project that is easiest to do badly and hardest to notice when it has been done badly, because the insecure version works perfectly well until the day it does not.

Aspect	Explanation in simple terms
What it is	Instance principals let a compute instance authenticate to OCI as itself. The instance obtains a short-lived certificate from a local metadata service; the OCI SDK inside the application requests and refreshes it automatically.
What it does in this project	When a pod needs to read or write an object in Object Storage, the SDK fetches a fresh certificate from the node and presents it. IAM checks which dynamic group that instance belongs to and which policies apply, and allows or denies the call.
Settings that were chosen	A dynamic group whose matching rule selects the worker instances in the production compartment; policies scoped to specific actions on specific resources — for example permission to manage objects in one named bucket, and permission to read images from the registry; nothing granted at tenancy level; no API keys, passwords or config-file credentials anywhere in the application or its images.

Aspect	Explanation in simple terms
Why it matters	Credentials that are never written down cannot leak, cannot be committed to a repository, cannot appear in a log, and never need a rotation project. Certificates are short-lived and rotate automatically.
What would break without it	You are back to static API keys living in configuration or environment variables — copied between environments, shared between services, and impossible to revoke confidently once they have spread.

What "least privilege" means in practice

- Grant verbs, not roles. Permission to manage objects in one bucket, rather than permission to administer Object Storage.
- Scope to a compartment, never to the tenancy. A rule written at tenancy level applies everywhere, including places nobody was thinking about when it was written.
- Name the resource. The policy identifies the specific bucket, not all buckets.
- Separate identities per purpose. The Go pool and the Python pool need different things and should not share one over-broad grant.
- The test to apply: if one worker node were completely compromised, what could the attacker reach? With this design, the answer is one bucket and the ability to pull images — not the account.

5.14 Object Storage

Aspect	Explanation in simple terms
What it is	Network file storage in which files are called objects and live in a bucket. Effectively unlimited, durable, and reached over the network by name.
What it does in this project	It holds everything that must outlive any individual pod or machine: user uploads, generated reports and exports, batch inputs and outputs, and backups.
Settings that were chosen	A private bucket in the production compartment, never public; access granted only to the dynamic group by IAM policy; versioning enabled so an accidental overwrite is recoverable; lifecycle rules to move older objects to cheaper storage tiers or delete them on a schedule; all access via the Service Gateway so it never traverses the internet; server-side encryption at rest by default.
Why it matters	Containers are disposable and their filesystems vanish when they stop. Anything that matters has to live somewhere else, and it has to be reachable identically from every replica.
What would break without it	Services would write to local disk. Two replicas would then hold different data, a restart would lose it, and the whole system could no longer be scaled horizontally.

5.15 Monitoring, logging and observability

Aspect	Explanation in simple terms
What it is	The systems that report what the platform and the applications are actually doing: the Kubernetes metrics server, OCI Monitoring for metrics and alarms, and OCI Logging for log

Aspect	Explanation in simple terms
	collection.
What it does in this project	The metrics server supplies the CPU and memory figures the pod autoscaler makes its decisions from. OCI Monitoring stores metrics and raises alarms. OCI Logging collects pod and node logs so an incident can be investigated after the fact.
Settings that were chosen	Alarms on error rate, latency, pod restart counts, pending pods and node saturation; dashboards per service; log retention appropriate to the environment; correlation identifiers passed through requests so one journey can be traced across services.
Why it matters	Autoscaling literally cannot work without metrics — the pod autoscaler has no input otherwise. And a system that changes itself automatically must be observable, or nobody can explain afterwards what it did.
What would break without it	The autoscaler is blind, alarms do not fire, and every incident becomes guesswork on a system that has been quietly rearranging itself.

6. The Architecture Diagram

The diagram on the next page shows every component described in Section 5 and how they connect. Section 6.2 then walks through the diagram element by element, from the outside inwards.

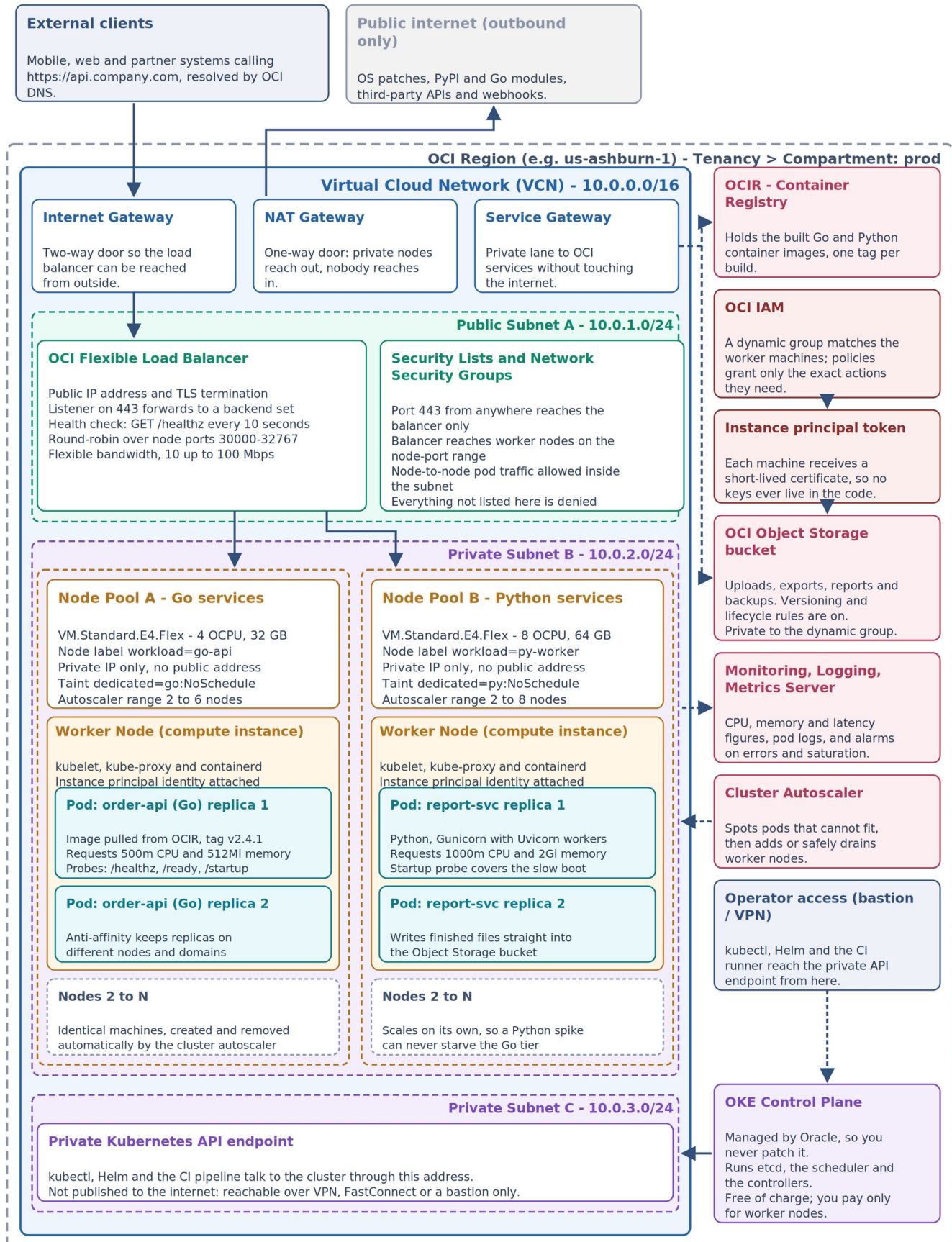
Diagram 2 - Full Architecture (every moving part)

Diagram 2 — the full architecture. Solid arrows are request paths; dashed arrows are control, identity and data paths.

A note on the drawing conventions used above:

- The outer dashed rectangle is the OCI region and the compartment inside it. Everything in the project lives here.
- The solid blue rectangle is the VCN — the private network.
- The dashed inner rectangles are subnets: green for the public one, purple for the private ones.
- The right-hand column holds regional services. They belong to the region rather than to the network, which is why they sit outside the VCN box.
- Solid arrows carry request traffic. Dashed arrows carry identity checks, image pulls, metrics and control instructions.

6.1 Reading the diagram in one minute

Traffic enters at the top left and travels downwards: clients → Internet Gateway → Load Balancer → worker nodes → pods. Everything on the right-hand side is a supporting service that the pods and nodes call out to. Everything inside the purple boxes is unreachable from the internet.

6.2 Every element of the diagram, explained

6.2.1 Outside the region (top of the diagram)

Element	What it represents and why it is drawn there
External clients	Mobile apps, web front-ends and partner systems. They resolve api.company.com through OCI DNS and receive the load balancer's public address. They have no knowledge of anything else in the diagram — which is the point.
Public internet (outbound only)	Drawn separately from the clients because traffic to it flows the other way. This is where operating-system patches, Python and Go packages, and third-party APIs come from. The arrow points outwards from the NAT Gateway, and there is deliberately no arrow coming back in.

6.2.2 The region and compartment (outer dashed box)

Everything inside this boundary lives in a single OCI region — one geographic location — and inside a single compartment, which is the labelled box used for access control and billing. Drawing the compartment explicitly matters, because every IAM policy in this project is scoped to it rather than to the whole tenancy.

6.2.3 The VCN and its three gateways

Element	What it represents and why it is drawn there
VCN 10.0.0.0/16	The private network holding every compute resource. The address range gives roughly 65,000 private addresses, comfortably more than the cluster will ever need, which leaves room to add subnets later without renumbering.
Internet Gateway	Drawn at the top because it is the only two-way door. The arrow from the clients arrives here and continues to the load balancer. No arrow from it reaches the private subnets — that absence is one of the most important facts in the diagram.
NAT Gateway	The one-way door. Its arrow points up and out to the public internet box, and nothing points back in. Private nodes use it to fetch packages and reach third-party APIs.

Element	What it represents and why it is drawn there
Service Gateway	The private lane to Oracle's own services. Its dashed arrow travels down the right-hand side to the container registry and to Object Storage, bypassing the internet entirely.

6.2.4 The public subnet

Element	What it represents and why it is drawn there
Public Subnet A — 10.0.1.0/24	The only zone with a route to the Internet Gateway. Note what is inside it: the load balancer, and nothing else. No application, no database, no worker node.
OCI Flexible Load Balancer	Holds the public IP, terminates TLS with a certificate held in OCI Vault, and forwards to a backend set of worker nodes on a node port. Its health check (GET /healthz every ten seconds) is what removes an unhealthy node from rotation, and it is the reason rolling deployments are invisible to users. Flexible bandwidth means it can absorb bursts without paying for peak capacity permanently.
Security Lists and NSGs	Drawn alongside the load balancer because they govern this boundary. The rules are deliberately narrow: port 443 inbound to the balancer only; balancer to nodes on the node-port range only; node-to-node traffic allowed for the pod network; everything else denied.

6.2.5 The private subnet with the node pools

Element	What it represents and why it is drawn there
Private Subnet B — 10.0.2.0/24	Where all the compute lives. There is no route from the Internet Gateway to this subnet, and the machines inside have no public IP addresses. The only inbound traffic is from the load balancer, on the permitted ports.
Node Pool A — Go services	Its box lists everything that defines the pool: the machine shape, the node label used to steer pods to it, the taint that keeps other pods off it, and the autoscaling range. Two separate pool boxes, side by side, is the visual form of workload isolation.
Node Pool B — Python services	The same structure with a larger shape and a wider scaling range, reflecting Python's heavier resource profile. Because the scaling ranges are separate, a Python surge can never consume the capacity the Go tier depends on.
Worker Node (compute instance)	One representative virtual machine per pool. The listed agents — kubelet, kube-proxy, containerd — are what make it a Kubernetes node rather than just a server. The instance principal identity attached to it is what lets the pods on it authenticate to OCI.
Pod boxes	Individual running copies. Each shows the image and tag it runs, the CPU and memory it reserves, and its health probes. Two replicas are drawn per pool to make anti-affinity visible: replicas of the same service are deliberately placed on different machines.
Nodes 2 to N	The dashed box that says the drawing is a sample, not the whole fleet. The real number changes minute by minute as the cluster autoscaler adds and removes machines. This box is the visual reminder that node count is an output of the system, not a setting.

6.2.6 The Kubernetes API endpoint subnet

Private Subnet C exists for one thing: the address that kubectl, Helm and the CI pipeline connect to in order to change the cluster. It is deliberately private. Administration therefore requires being on the corporate network — via VPN, FastConnect or a bastion host — which means a leaked kubeconfig file is not, on its own, enough to reach the cluster.

6.2.7 The regional services column

Element	What it represents and why it is drawn there
OCIR — Container Registry	The private image warehouse. The dashed arrow from the Service Gateway shows that image pulls travel on Oracle's backbone rather than over the internet.
OCI IAM	Holds the dynamic group that matches the worker instances, and the policies that state exactly what those instances may do. Drawn above the instance-principal box because it is what gives that identity its meaning.
Instance principal token	The short-lived certificate each machine obtains locally. Its arrow points down to Object Storage: this is the identity being used to authorise the actual data call.
OCI Object Storage bucket	The private bucket. Versioning protects against accidental overwrites; lifecycle rules control cost; access is restricted to the dynamic group, so no human credential and no other service can reach it.
Monitoring, Logging, Metrics Server	The arrow points from the nodes outwards, because this is telemetry being shipped rather than a request being served. The metrics server also feeds the pod autoscaler.
Cluster Autoscaler	Its dashed arrow points back into the private subnet, because unlike everything else in the column it acts on the cluster: it adds and drains worker nodes.
Operator access (bastion / VPN)	How engineers and the CI pipeline reach the private API endpoint. Its arrow leads to the control plane, not to the pods — nobody logs into a machine to deploy anything.
OKE Control Plane	Oracle-managed and drawn deliberately outside the VCN, because you never patch it, size it or pay for it. Its arrow points to the private API endpoint, which is the cluster's only administrative entrance.

6.3 What the diagram is designed to prove

- There is exactly one way in. Port 443, to the load balancer, and nowhere else.
- Nothing that runs your code is reachable from the internet. Every application pod sits behind a private subnet with no inbound route.
- Data does not travel over the internet. Object Storage and registry traffic goes through the Service Gateway on Oracle's backbone.
- There are no stored credentials anywhere on the diagram. The only identity is a short-lived certificate issued to the machine itself.
- The number of machines is not a fixed decision. Two boxes in the diagram — the autoscaler and "Nodes 2 to N" — exist to make that explicit.

7. Zooming In: What Actually Runs on One Machine

Diagram 2 shows the whole system. This section zooms into a single worker node, because that is where the abstract terms — pod, container, probe, resource request — become concrete.

Diagram 3 - Zoom In: Inside One Worker Node

A worker node is just a virtual machine. Kubernetes places pods on it, and a pod is a wrapper around your container.

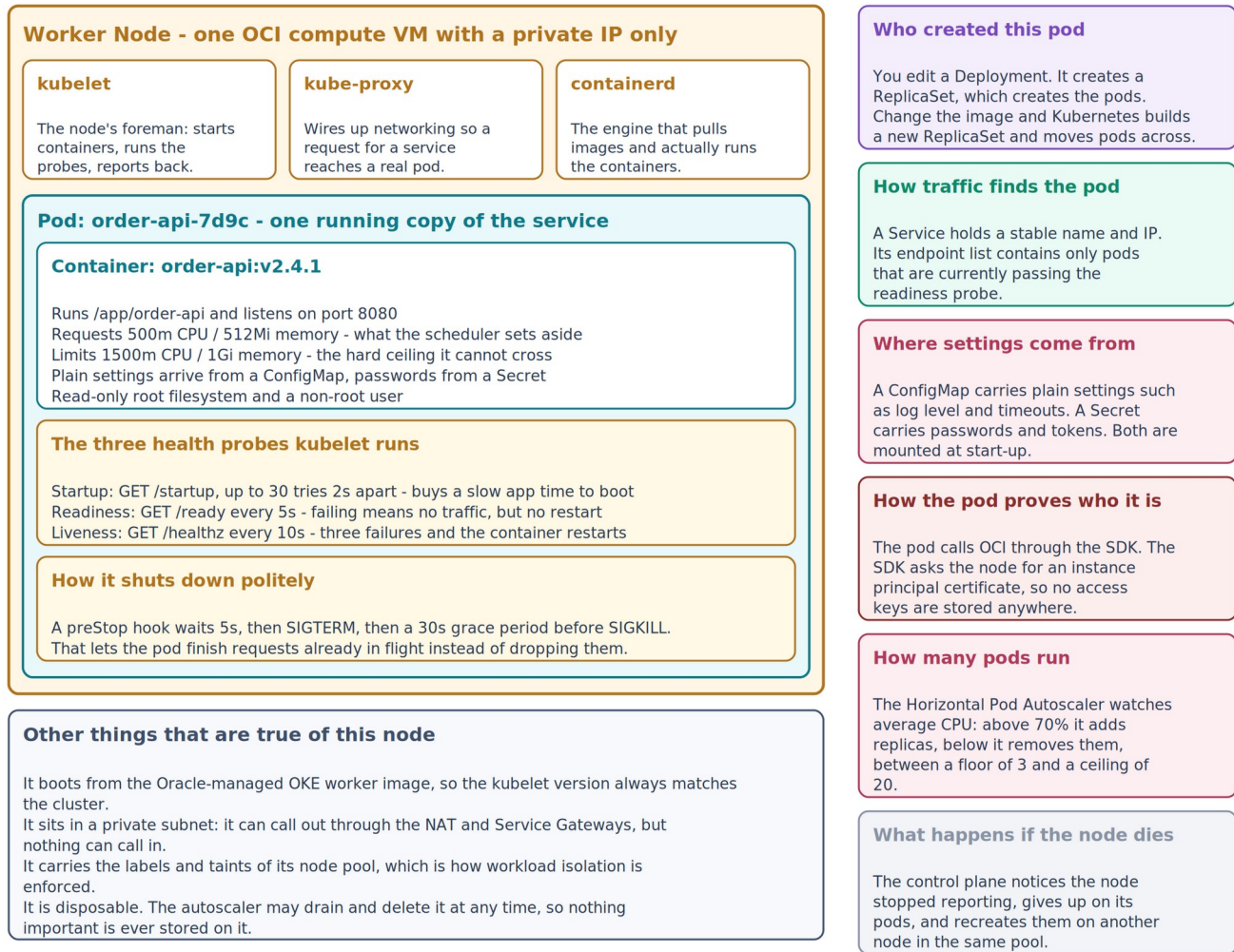


Diagram 3 — one worker node, one pod, one container, and the objects that govern them.

7.1 The layers, from the outside in

- The node is a virtual machine. It has cores, memory, a disk and a private IP address. Nothing about it is special until three agents are running on it.
- kubelet is the local foreman. It receives instructions from the control plane, starts and stops containers, runs the health probes on a schedule and reports status back. If the kubelet stops reporting, the control plane concludes the node is gone.
- kube-proxy handles local networking, so that traffic addressed to a Service reaches a pod that is actually alive and ready.
- containerd is the runtime that pulls images from OCIR and runs the containers themselves.

10. The pod is the unit Kubernetes schedules. Here it wraps one container, and it is what carries the labels, the resource requests, the probes and the scheduling rules.
11. The container is the running program: the Go or Python process, listening on a port, doing the work.

7.2 Requests and limits — the most consequential two numbers

Every container declares two pairs of numbers, and almost every capacity problem in a Kubernetes cluster traces back to them being wrong.

Setting	What it actually does
Requests	What the scheduler sets aside for this container. It is the number used to decide whether the pod fits on a node. Set it too low and machines become oversubscribed and unpredictable under load; set it too high and you pay for reserved capacity nobody uses.
Limits	The hard ceiling. Exceeding the CPU limit causes the container to be throttled — slowed down. Exceeding the memory limit causes it to be killed outright and restarted, because memory cannot be throttled.
Why the gap between them matters	A modest gap lets a container use spare capacity during a short burst without reserving it permanently. A very large gap invites a node to become badly oversubscribed, because the scheduler only ever counts requests.

7.3 The three probes on a real timeline

Take a Python pod that needs eight seconds to import its dependencies and open its connections:

- Seconds 0–8: the startup probe polls /startup and receives failures. This is expected. Readiness and liveness are suspended, so nothing restarts the pod and no traffic is sent to it.
- Second 8: the startup probe succeeds. It never runs again. Readiness and liveness now begin.
- Second 9: the readiness probe succeeds. The pod is added to the Service and, through it, becomes eligible to receive live traffic.
- Steady state: readiness runs every five seconds, liveness every ten. Both keep passing.
- Later, a dependency becomes briefly unavailable and readiness starts failing. The pod is removed from the Service — it stops receiving traffic — but it is not restarted, because liveness still passes. When the dependency recovers, readiness passes again and the pod rejoins automatically. No human involved.
- A different failure: the process deadlocks and stops responding at all. Liveness fails three times in a row and the container is restarted.

7.4 Why nothing important is stored on the node

The last box in Diagram 3 states that the node is disposable, and this is a design rule rather than an observation. The cluster autoscaler may drain and delete any node at any time; a node may fail; a pod may be evicted and rescheduled elsewhere in seconds. Because of that, application state lives in Object Storage or a database, and the container's own filesystem is treated as scratch space — read-only where possible. This single rule is what makes horizontal scaling, rolling updates and automatic recovery all work.

8. Workflows, End to End

The sections so far described the parts. These describe the parts in motion. Each workflow is written as a sequence of steps, in the order they actually happen.

8.1 Workflow 1 — one user request, from tap to response

Diagram 4 - One Request, End to End

What happens between a customer tapping a button and the answer coming back.

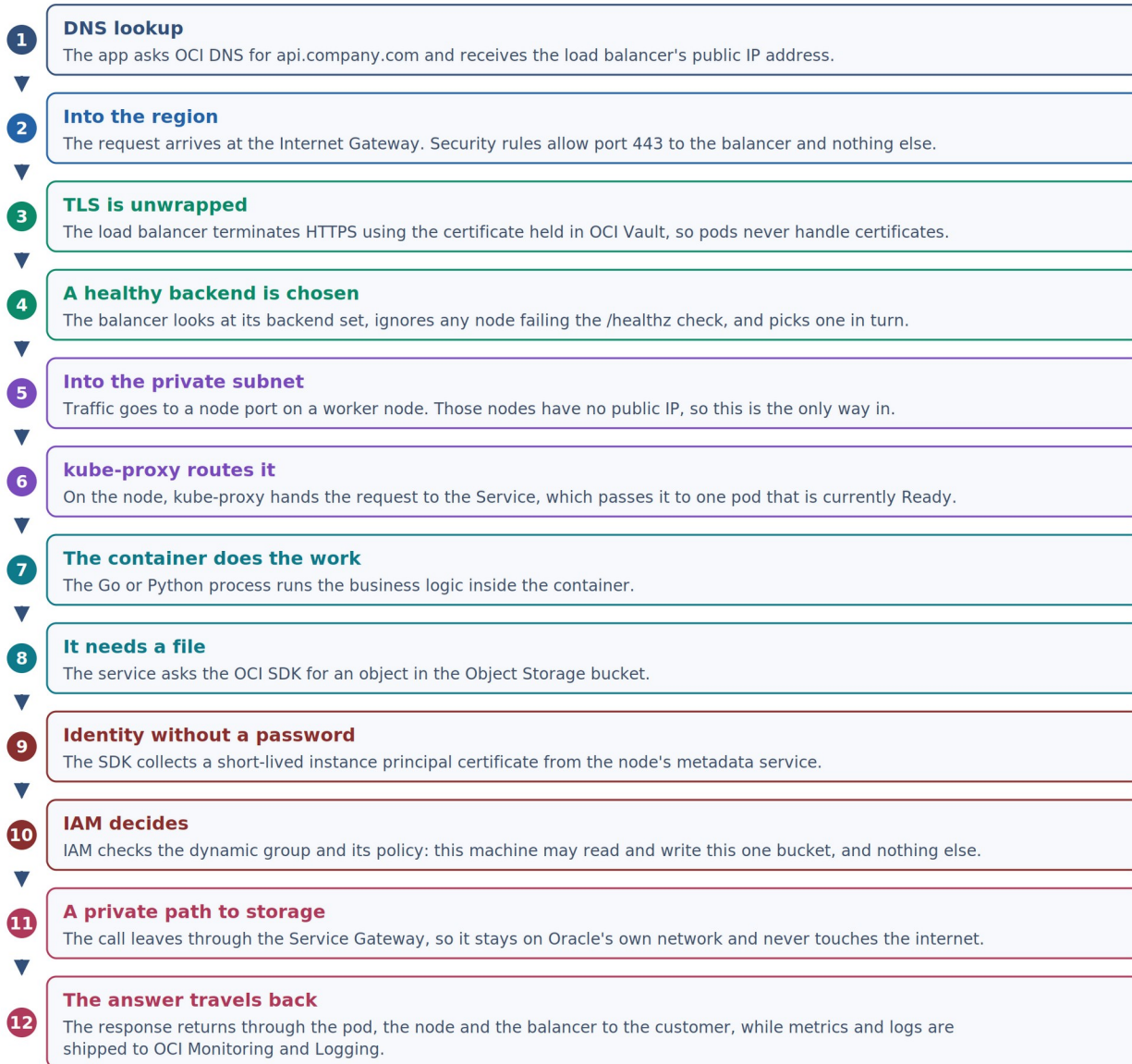


Diagram 4 — the twelve steps of a single request.

The same twelve steps, with the detail

#	Step	What happens in detail
1	DNS resolution	The client resolves api.company.com through OCI DNS and receives the load balancer's

#	Step	What happens in detail
		public IP address. Nothing else in the system has a public address, so this is the only answer DNS can give.
2	Arrival at the region	The request reaches the Internet Gateway on port 443. Security rules permit that port to the load balancer and deny everything else, so any other probe or port scan stops here.
3	TLS termination	The load balancer decrypts the request using the certificate held in OCI Vault. From this point the traffic travels in plaintext, but only inside the private network.
4	Backend selection	The balancer consults its backend set. Any node failing the GET /healthz check has already been removed from rotation. It selects one of the healthy nodes, round-robin.
5	Entry to the private subnet	The request is forwarded to a node port on the chosen worker node. Because that node has no public IP, this path from the load balancer is the only way traffic can reach it at all.
6	Local routing	kube-proxy on the node passes the request to the Service, which forwards it to one pod drawn from its endpoint list. That list contains only pods currently passing their readiness probe, so a pod that is starting, shutting down or unhealthy is never chosen.
7	Application logic	The Go or Python process handles the request inside its container: validation, business rules, calls to other internal services if needed.
8	A file is needed	Suppose the request needs an object from the bucket. The application calls the OCI SDK, which needs to authenticate.
9	Identity is obtained	The SDK contacts the instance metadata service on the node and receives a short-lived instance principal certificate. No key, password or token was read from a file, an environment variable or the image.
10	Authorisation is checked	IAM evaluates the request: this instance is a member of the worker dynamic group, and a policy grants that group permission to manage objects in this one named bucket. The call is allowed. A call to any other bucket would be denied on the same evidence.
11	The private data path	The call to Object Storage leaves through the Service Gateway, travelling across Oracle's backbone. It never touches the public internet, which is faster, cheaper and less exposed than the NAT route.
12	The response returns	The object is returned to the pod, the pod builds the response, and it travels back through the node and the load balancer to the client. In parallel, metrics and logs are shipped to OCI Monitoring and Logging, and the CPU figures from this request contribute to the data the pod autoscaler will act on.

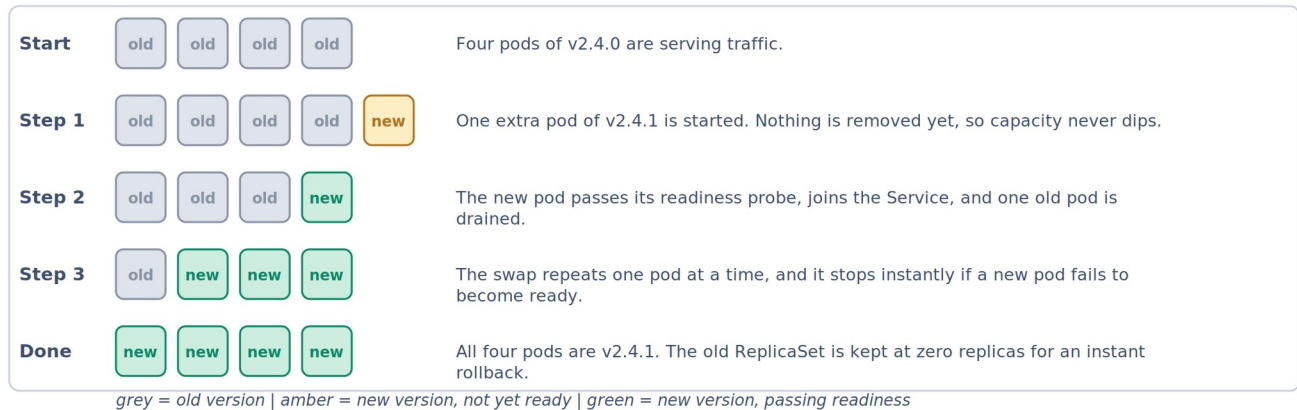
What that whole journey never involved

No engineer, no stored password, no public exposure of any machine running application code, and no knowledge on the client's part of how many copies exist. Any of the pods could have served the request, and if the chosen one had died mid-flight the client would have been retried against another.

8.2 Workflow 2 — building and releasing a new version

Diagram 5 - Shipping a New Version, and Growing Under Load

A. Rolling deployment (surge 1, unavailable 0): four pods, no downtime



B. Two levels of autoscaling, working one after the other

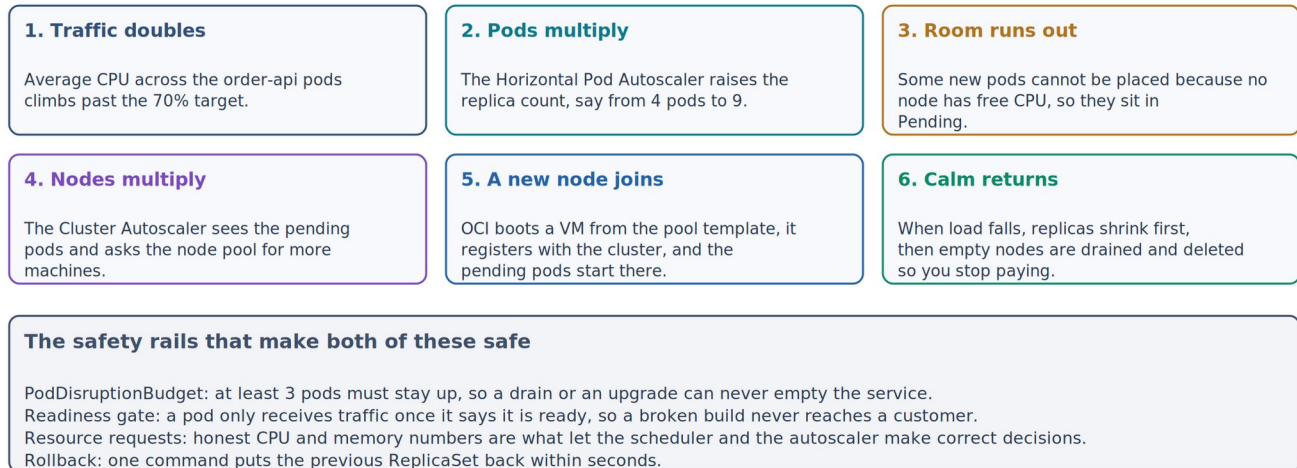


Diagram 5 — the rolling deployment, and the two levels of autoscaling.

The build and release sequence

12. A developer merges a change. The CI pipeline starts automatically.
13. Tests run. If they fail, the pipeline stops here and nothing reaches the cluster.
14. A container image is built using a multi-stage build, so compilers and build tooling do not end up in the final image.
15. The image is tagged with the exact version or commit — never a moving tag — and pushed to OCIR.
16. The pipeline connects to the private Kubernetes API endpoint through the bastion or VPN path and applies the new image tag to the Deployment. This is the entire deployment action: change one value.
17. Kubernetes creates a new ReplicaSet for the new version and begins the rolling update described below.
18. The rollout is watched. If it does not complete within the progress deadline, it is marked failed and can be rolled back with a single command, because the previous ReplicaSet is still there at zero replicas.

What the rolling update does, step by step

Stage	What happens
Start	Four pods of the current version are serving traffic behind the Service.
Surge	Because maxSurge is 1, one extra pod is created running the new version. Because maxUnavailable is 0, nothing is removed yet — capacity never dips below four.
Wait for ready	The new pod starts, passes its startup probe, then its readiness probe. Only at that moment is it added to the Service and the load balancer's rotation. If it never becomes ready, the rollout stops here and every request is still served by the old version.
Retire one	One old pod is now removed. It is sent a termination signal, its preStop hook runs, and it is given a grace period to finish the requests already in flight before being stopped.
Repeat	The surge-and-retire cycle repeats one pod at a time until every pod runs the new version.
Finish	The old ReplicaSet is kept, scaled to zero. Rolling back is therefore a matter of scaling it up again — seconds, not a rebuild.

8.3 Workflow 3 — a traffic spike, handled automatically

19. Traffic doubles. Requests per second climb sharply across the Go services.
20. CPU rises. Average CPU utilisation across the order-api pods passes the 70% target.
21. Pods are added. The Horizontal Pod Autoscaler increases the replica count — for example from four to nine. New pods start in seconds, pass their probes, and begin taking traffic.
22. Capacity runs out. Some of the new pods cannot be placed, because no existing node has enough unreserved CPU to satisfy their requests. Those pods sit in Pending.
23. Machines are added. The Cluster Autoscaler notices the Pending pods and asks Node Pool A for more machines, within its configured maximum.
24. A node joins. OCI provisions a virtual machine from the pool template. It boots, the kubelet registers it with the cluster, and the scheduler immediately places the Pending pods onto it.
25. Traffic subsides. The pod autoscaler reduces replicas first, after a stabilisation window so that a brief dip does not cause thrashing.
26. Machines are removed. Nodes that stay underused for a sustained period are cordoned, drained — respecting the PodDisruptionBudget so the service never drops below its minimum — and then deleted. Billing stops for them.

The important property here

Nobody was paged, nobody logged in, and no capacity decision was made in advance. The only human input was choosing the thresholds and the upper and lower bounds — and the bounds are what stop a runaway loop from scaling into a very large invoice.

8.4 Workflow 4 — a machine dies at three in the morning

27. A worker node fails. Its kubelet stops reporting to the control plane.
28. After a grace period, the control plane marks the node NotReady and stops scheduling anything new onto it.

29. The load balancer's health check against that node fails, so it is removed from the backend rotation. Client traffic stops being sent there within seconds.
30. The pods that were running on that node are declared lost. Their ReplicaSets notice the count is short and create replacements.
31. The scheduler places the replacements on other nodes in the same pool. If there is not enough room, the cluster autoscaler adds a machine — the same mechanism as a traffic spike.
32. The new pods pass their startup and readiness probes and join the Service. Full capacity is restored.
33. Because anti-affinity had spread the replicas across machines in the first place, the service never lost all its copies, and users saw at most a brief blip on in-flight requests.

The whole sequence is automatic. The morning's work is reading the incident afterwards, not restoring the service.

8.5 Workflow 5 — how a pod gets permission to touch data

34. The application code calls the OCI SDK. It contains no credentials of any kind.
35. The SDK detects that it is running on an OCI compute instance and requests an instance principal certificate from the instance metadata service, which is only reachable from the machine itself.
36. The certificate is returned. It is short-lived and is refreshed automatically by the SDK before it expires.
37. The SDK signs the API call with that certificate and sends it to Object Storage.
38. IAM resolves the caller's identity to a compute instance, checks which dynamic groups that instance matches, and evaluates the policies attached to those groups.
39. A policy states that the worker dynamic group may manage objects in one named bucket in the production compartment. The call matches, and is allowed.
40. Any call outside that grant — a different bucket, a different service, an administrative action — is denied at this step, with no additional configuration required.
41. The request travels to Object Storage through the Service Gateway, staying on Oracle's network throughout.

Why this is the design decision worth defending in an interview

Every alternative involves a long-lived secret existing somewhere: in an image, in an environment variable, in a config file, in a CI system. Each of those places is somewhere it can leak, and none of them makes rotation easy. Instance principals remove the secret entirely rather than protecting it better.

8.6 Workflow 6 — upgrading the cluster without downtime

42. Oracle upgrades the managed control plane. This is transparent — the applications do not notice.
43. For worker nodes, a new node pool is created on the new Kubernetes version, or the existing pool's image is updated.
44. New nodes join the cluster on the new version and become schedulable.
45. Old nodes are cordoned one at a time, so nothing new is scheduled onto them.
46. Each old node is drained. Its pods are evicted politely, honouring the PodDisruptionBudget, so the service never drops below its minimum number of replicas.
47. Evicted pods are rescheduled onto the new nodes and must pass their probes before receiving traffic.
48. Once empty, the old node is deleted. The sequence repeats until the whole pool is on the new version.
49. Because the pools are separate, the Go pool and the Python pool can be upgraded on different days, so a problem affects one workload family rather than everything.

8.7 Workflow 7 — changing a setting without rebuilding anything

50. A configuration value needs to change — a timeout, a log level, a feature flag.
51. The value lives in a ConfigMap, not baked into the image, so no rebuild is required.
52. The ConfigMap is updated and the Deployment is restarted, which triggers exactly the same rolling update mechanism as a code release.
53. Pods pick up the new value as they are recreated, one at a time, with no downtime.
54. Sensitive values follow the same path but live in a Secret rather than a ConfigMap, and are mounted separately.

The same principle applies throughout: there is one mechanism for changing what is running — update the desired state and let the platform converge on it — and every kind of change goes through it.

9. Design Decisions and Their Trade-Offs

Every choice below had a reasonable alternative. Being able to say why the alternative was rejected is usually more valuable than the choice itself.

Decision	The alternative that was rejected	Why this way
Managed OKE rather than self-managed Kubernetes	Running the control plane yourself on virtual machines.	The control plane is a permanent operational burden — high-availability etcd, certificate rotation, upgrades — and Oracle does not charge for it. Self-managing buys control that this project had no need for.
Separate node pools per language	One general-purpose pool for everything.	The two workloads have very different resource profiles. Separate pools give independent scaling, independent upgrades and genuine isolation, at the cost of a slightly less densely packed cluster.
Worker nodes in private subnets	Public IPs on nodes for easier direct access.	It removes an entire class of attack. The extra work is a bastion or VPN for administration, which is a small and well-understood cost.
Instance principals	API keys or passwords in configuration or environment variables.	No long-lived secret exists to leak or rotate. The trade-off is that it only works on OCI compute — but that is exactly where this workload runs.
Service Gateway for OCI traffic	Routing everything through the NAT Gateway.	Faster, cheaper (no NAT data processing charges) and never exposed to the internet. The cost is one more route rule to configure and understand.
TLS terminated at the load balancer	End-to-end TLS terminated inside each pod.	Certificates are managed in one place and renewals need no redeployment, while the cryptographic work is handled by the managed service. If regulation demands encryption on the internal hop too, this is the decision to revisit.
maxUnavailable 0 during rollouts	Allowing capacity to dip to release faster.	Full capacity is maintained throughout the release, at the cost of one extra pod's resources for the duration.
Autoscaling with a floor of three replicas	Scaling down to one replica when idle.	Three replicas guarantee redundancy across machines at all times. It costs slightly more at idle and removes a whole category of avoidable incident.
Object Storage rather than local disk	Writing files to the node's filesystem.	Pods and nodes are disposable, so local disk data is lost data. Object Storage is reachable identically from every replica, which is what makes horizontal scaling possible at all.

9.1 Honest limitations of this design

A description that presents no drawbacks is not a description, it is a sales pitch. The genuine trade-offs here are:

- Complexity. Kubernetes has a real learning cost. For a small service with steady traffic, a couple of virtual machines behind a load balancer would be simpler and perfectly adequate.
- Single region. Everything described lives in one OCI region. That survives a machine failure and a data-centre (availability domain) failure, but not the loss of an entire region. A second region would require active work.
- Cold-start latency at the machine level. Adding pods takes seconds; adding machines takes minutes. A very sudden spike can outrun the second level of scaling, which is why the pod floor and the node minimum are set above what idle traffic requires.
- Cost visibility. Elastic infrastructure makes it easy to stop noticing spend. The maximum node counts and lifecycle rules on the bucket exist partly as financial guard rails.
- Probe quality is load-bearing. Almost every safety property in this document depends on the health probes telling the truth. A probe that always returns 200 quietly disables the protection it appears to provide.

10. Quick Answers and a One-Page Summary

10.1 Questions you are likely to be asked

Question	A short, honest answer
Why Kubernetes rather than just virtual machines?	Because the service needs many interchangeable copies that are created, replaced and moved constantly. Doing that by hand means writing your own scheduler, health checker and deployment tool. Kubernetes is that tool, already written and widely understood.
Why both Go and Python?	Go for the high-traffic latency-sensitive paths, because it is compiled, fast and light on memory. Python where library support and development speed matter more. Containers make the mixture nearly free, and separate node pools keep their different resource appetites from colliding.
What does the load balancer actually do for you?	It gives one public address for many copies, terminates TLS in one place, and — most importantly — removes unhealthy backends automatically, which is what makes rolling deployments and node failures invisible to users.
Difference between liveness and readiness?	Readiness failure means stop sending traffic but leave the pod alone. Liveness failure means the pod is beyond saving, so restart it. Getting them the wrong way round either serves requests to a broken pod, or restarts healthy pods every time a dependency has a bad minute.
What are the two kinds of autoscaling and why both?	The pod autoscaler adds copies within existing machines — fast, seconds. The cluster autoscaler adds machines when the copies no longer fit — slower, minutes. Pods first absorb the spike; machines second stop it hitting a ceiling.
How do the pods authenticate without secrets?	Instance principals. The machine obtains a short-lived certificate locally; IAM resolves it to a dynamic group and applies policies scoped to one bucket in one compartment. No key exists anywhere to leak or rotate.
What is workload isolation and how is it enforced?	Node labels steer pods to the right pool, taints stop anything else landing there, anti-affinity spreads replicas across machines, and resource requests and limits cap what any container can consume. Scheduler-enforced, not convention.
How would you make this survive a region outage?	It does not today, and that is the honest answer. It would need a second region with replicated storage, DNS-level failover and a data-consistency strategy — which is a substantially larger project than what is described here.
What is the weakest point of this design?	The health probes. Nearly everything else depends on them being truthful. A probe that returns success unconditionally silently disables rolling-update safety, load-balancer rotation and automatic restart all at once.

10.2 The whole project on one page

Layer	One-sentence summary
The applications	Small independent Go and Python services, each packaged as a versioned container image and stored in OCIR.

Layer	One-sentence summary
The orchestrator	OKE — Oracle's managed Kubernetes — keeps the right number of copies running, replaces failures and coordinates version changes; Oracle runs the control plane.
The machines	OCI Compute virtual machines organised into two node pools, one per language, sized to fit each workload and holding no public IP addresses.
The network	A VCN split into a public subnet for the load balancer and private subnets for everything else, with three gateways serving three different directions of travel.
The front door	One OCI Load Balancer terminating TLS, health-checking backends and spreading traffic across every healthy copy.
Elasticity	Two layers of autoscaling — pods first, machines second — bounded by minimums and maximums that protect both availability and the invoice.
Safe change	Rolling deployments with surge 1 and unavailable 0, gated by readiness probes, with the previous version retained for instant rollback.
Self-healing	Startup, readiness and liveness probes, plus the reconciliation loop that recreates anything lost and reschedules it elsewhere.
Isolation	Node labels, taints and tolerations, pod anti-affinity, and resource requests and limits on every container.
Identity	Instance principals with short-lived certificates and least-privilege IAM policies scoped to one compartment and one named bucket.
Persistence	OCI Object Storage, private, versioned and lifecycle-managed, reached over the Service Gateway so it never touches the internet.
Observability	OCI Monitoring, OCI Logging and the Kubernetes metrics server — which is also what makes autoscaling possible at all.

If you remember one sentence from this document

Everything running is disposable and replaceable, and every feature described here — health probes, rolling deployments, autoscaling, node pools, instance principals — exists to make that disposability safe, automatic and invisible to the person using the service.

Appendix A.1 — The Complete Architecture, Full Width

The same architecture as Diagram 2, redrawn full width for printing or presenting. Read it left to right: the private network and everything running inside it on the left, the Oracle-managed services it depends on down the right-hand side.

Figure 1 — End-to-End Architecture: Go & Python Microservices on Oracle Kubernetes Engine (OKE)

Arrows: orange = user traffic, identity & deployments • blue = cluster control traffic • green = private access to OCI services • grey dashed = machines supplied

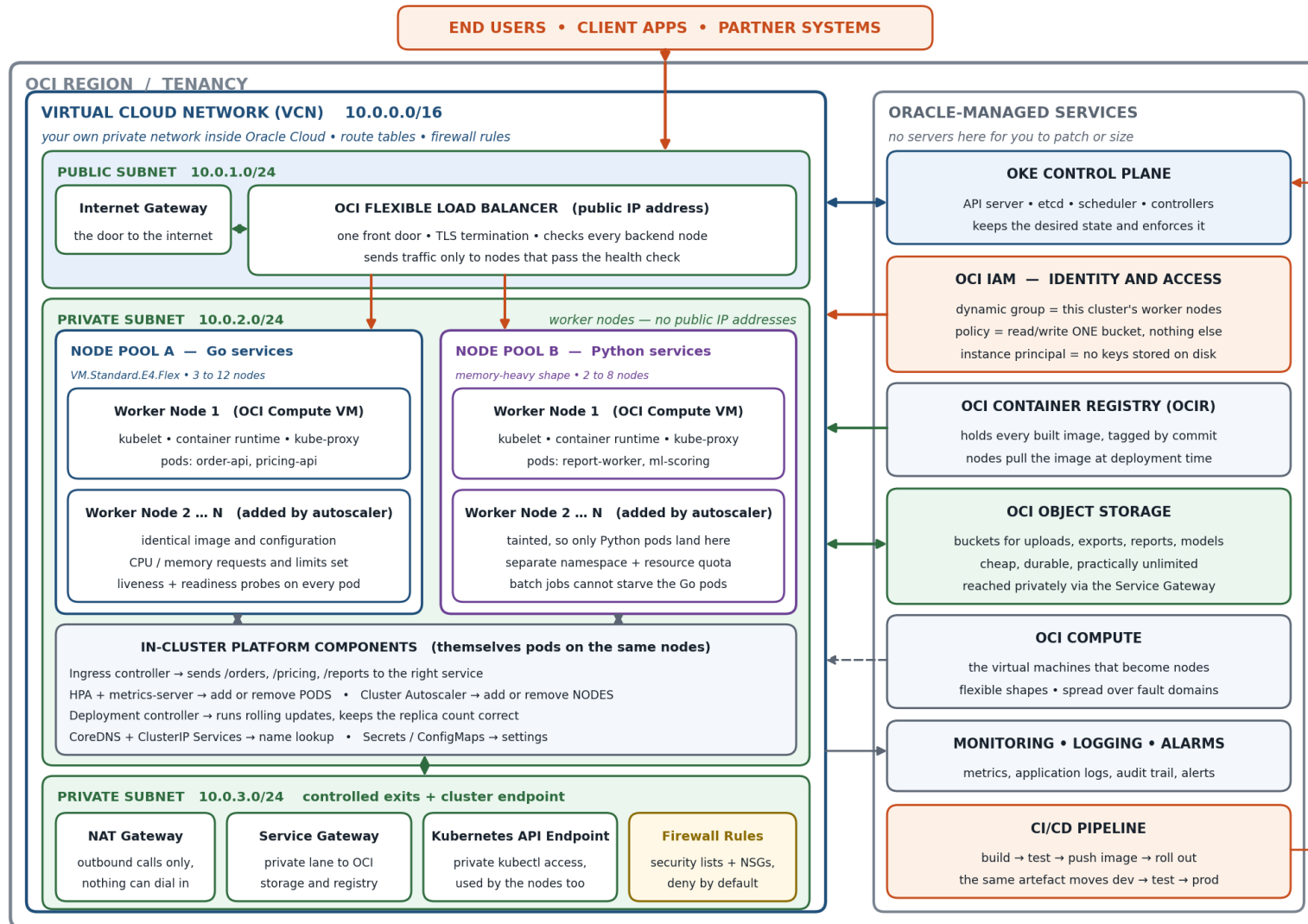


Figure A1 — the region, the VCN, its three subnets, the two node pools, and the managed services around them.

Appendix A.2 — One Request, Step by Step

The twelve numbered steps of Section 8.1 in a single picture. Follow the numbers: every hop is either a security check, a routing decision, or a piece of real work.

Figure 2 — What Happens to a Single User Request, Step by Step

Follow the numbers. Every hop is either a security check, a routing decision, or a piece of work.

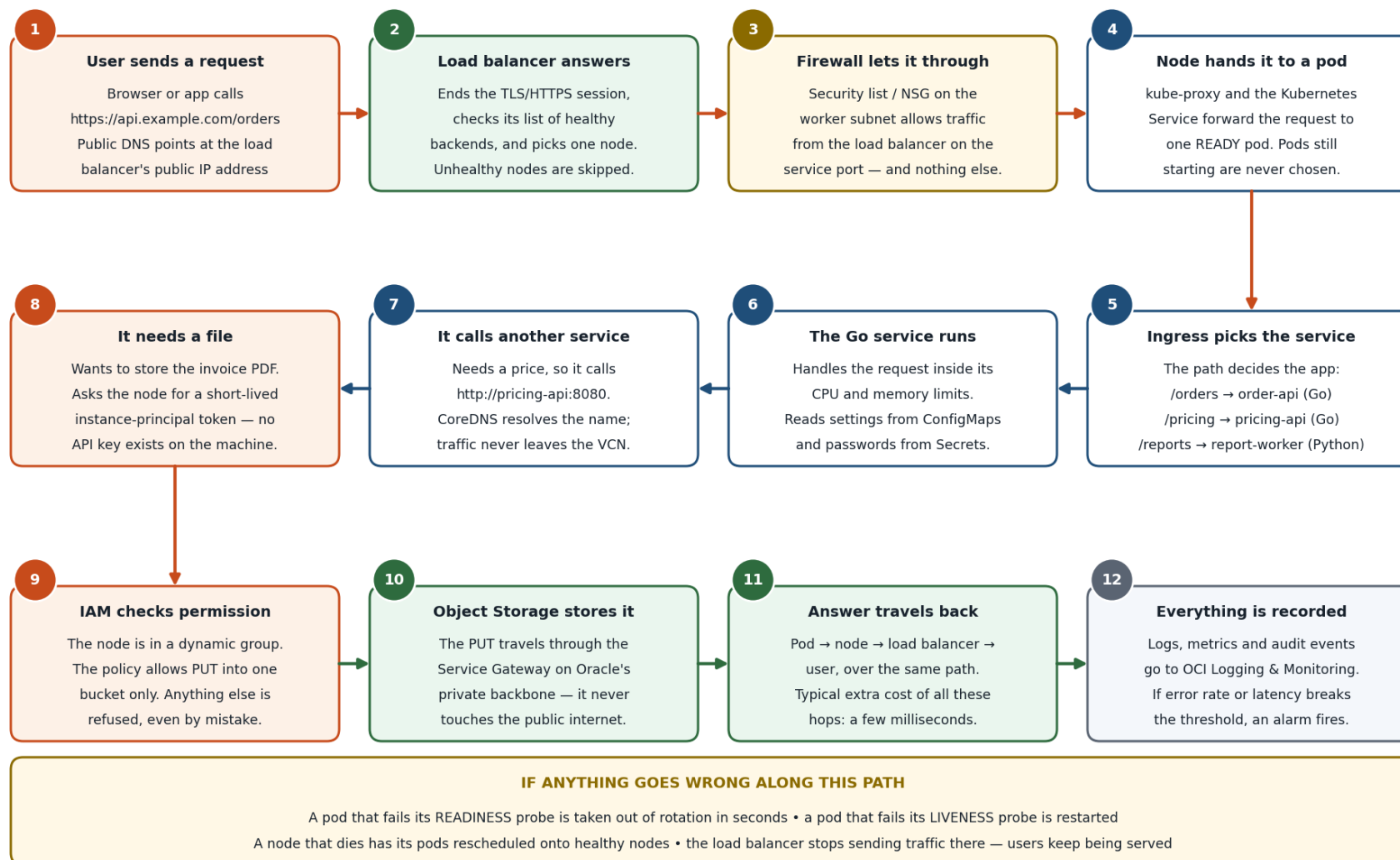


Figure A2 — from the user's tap to the answer coming back, including the identity check and the private path to Object Storage.

Appendix A.3 — From Commit to Live Traffic

The release pipeline across the top; underneath it, the rolling deployment shown one pod at a time. Green pods are the old version, blue pods the new one.

Figure 3 — From a Developer's Commit to Live Traffic (CI/CD + Rolling Deployment)

Nothing is edited on a running server. A new image is built once, then the cluster is asked to move to it — a few pods at a time.

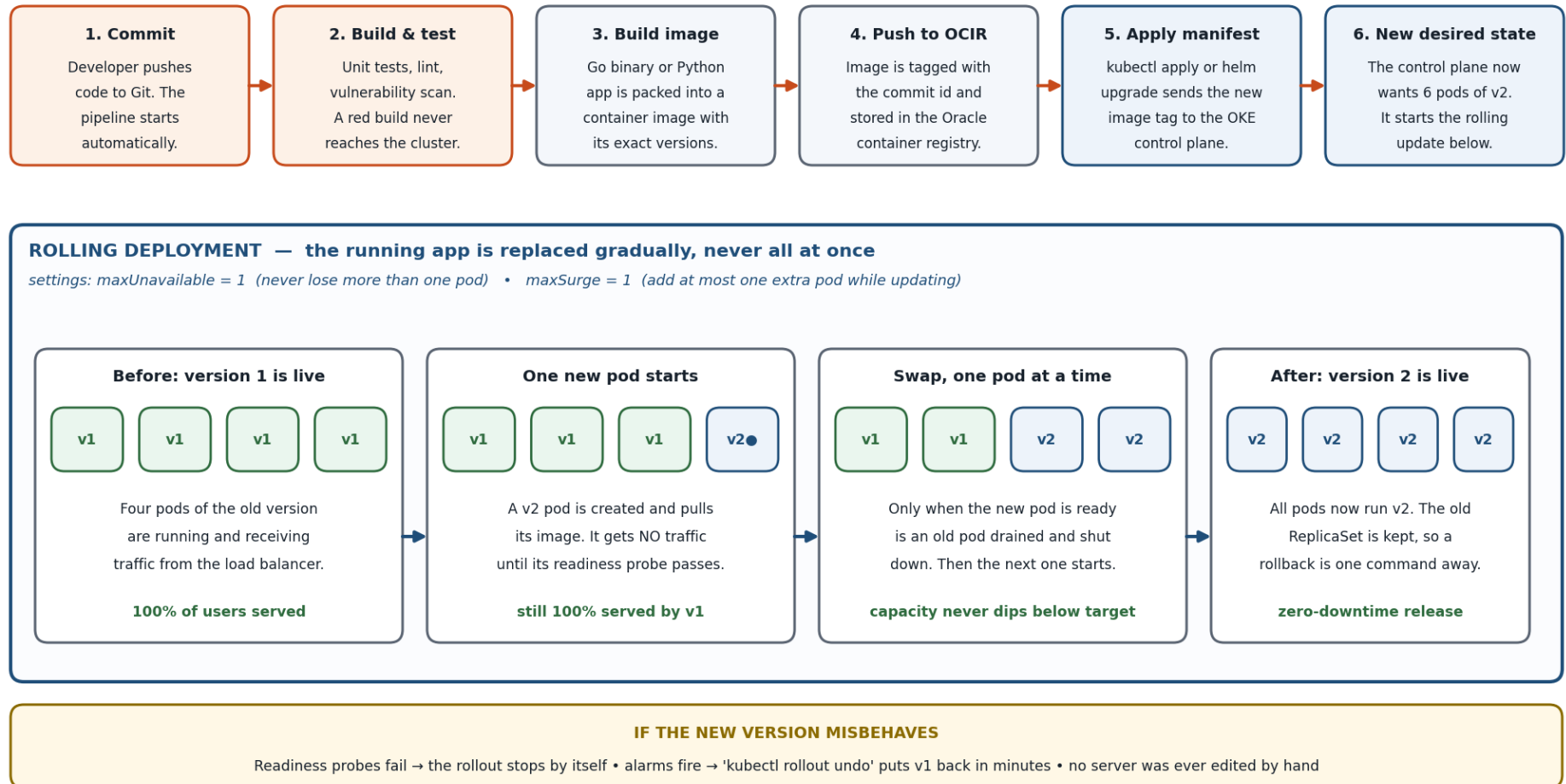


Figure A3 — the six pipeline stages, then the four stages of a zero-downtime rolling update.

Appendix A.4 — Autoscaling and Self-Healing

The two things the platform does without being asked: it changes its own size when demand changes, and it repairs itself when something breaks.

Figure 4 — How the Platform Reacts on Its Own: Autoscaling and Self-Healing

Nobody is paged for these. The cluster is told what 'good' looks like, and it keeps steering back to it.



Figure A4 — the two autoscalers working in sequence on the left; the automatic responses to five kinds of failure on the right.